

Lymphocyte Annotator: CD3⁺ and CD8⁺ IHC Stained Patch Image Annotation Tool

Muhammad Mohsin Zafar

Pattern Recognition Lab, DCIS
Pakistan Institute of Engineering and Applied Sciences
Islamabad, Pakistan
m.mohsin.zafar@outlook.com

Anabia Sohail

Pattern Recognition Lab, DCIS
Pakistan Institute of Engineering and Applied Sciences
Islamabad, Pakistan
anabia.sohail@gmail.com

Zunaira Rauf

Pattern Recognition Lab, DCIS
Pakistan Institute of Engineering and Applied Sciences
Islamabad, Pakistan
zunaira_18@pieas.edu.pk

Asifullah Khan

PIEAS Artificial Intelligence Center (PAIC)
Pattern Recognition Lab, DCIS
Pakistan Institute of Engineering and Applied Sciences
Islamabad, Pakistan
asif@pieas.edu.pk

Abstract— In digital pathology, preliminary step for the development of Computer Aided Diagnostics involves defining and labelling an Object of Interest. Contrary to traditional tiresome method of observation and marking of objects under microscope, digital pathology enables the use of annotation tools to capture localization information and morphology for targeted object(s). However, it is quite difficult to capture this information due to high inter and intra class variance. In this paper, we have addressed these two different problems by developing an easy to use and efficient tool “Lymphocyte Annotator” that allows a user to draw bounding box(es) on IHC stained images, and as a result, automatically generates multiple variants of segmentation masks for the targeted object. Lymphocyte Annotator provides multiple options for bounding box selection, with variation in bounding box size and type. The developed tool targets labelling of lymphocytes in histopathological images. This annotator generates five different types of masks by using a) Chan Vese Segmentation Model, b) Morphological Active Contours without Edges Model, c) Morphological Geodesic Active Contours Model, d) Adaptive Thresholding and e) Circular Masks, thus providing user with an opportunity to use a mixture of different types of generated masks. Capturing of lymphocytes’ morphology using different models incorporates morphological variance and enables training of robust deep Convolutional Neural Network. The annotations created from this tool are by default exported in PASCAL VOC and JSON format. Lymphocyte annotator is an open-source tool, and can be accessed from source, available at https://github.com/m-mohsin-zafar/lysto_labeltool.

Keywords— Lymphocytes; Annotation Tool; Computational Pathology; Deep Learning; Histopathological Images;

I. INTRODUCTION

With rising interest of community and different field experts in the applied Machine Learning and Deep Learning, the focus of world has shifted towards “Data” as the new currency. Machine Learning and Deep Learning models require huge amounts of processed data to be able to make sense out of it and consequently perform tasks such as pattern recognition, detection, classification etc. Data is a very tough commodity to come by in many fields and specifically in “computational pathology”. In computational pathology,

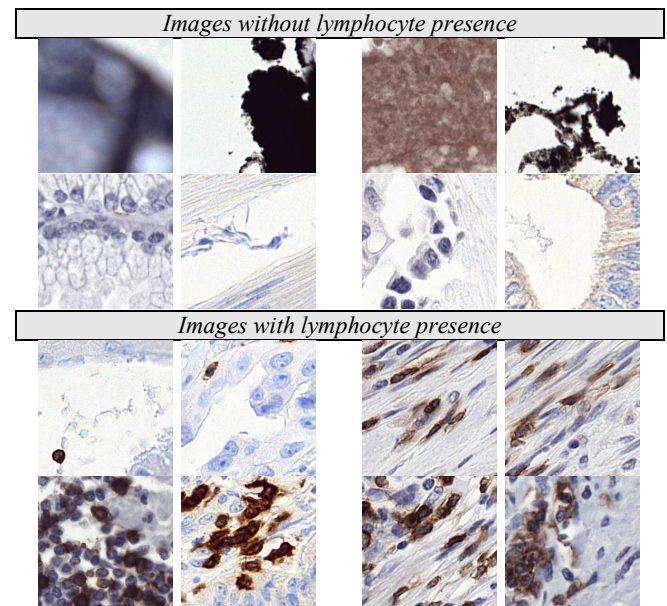


Fig. 1 Sample Images (*Images without lymphocyte contain both artifacts (row 1) and normal tissue regions (row 2), Images with lymphocyte presence contain sparse as well as dense bodies without a generalized morphology*)

development of Machine Learning based diagnostic systems for automation of tumor assessment, immune score, nuclear and cellular bodies’ detection/segmentation, etc. are the emerging areas of research. Large amount of pathological data enables the utilization of computer vision, deep learning, and other pattern recognition methods in designing automated diagnostic systems. However, the two main problems that persist in the field of computational pathology are, (a) the inception of raw data in large amount, and (b) the labelling or rather agreed upon labelling of data. The reasons for both the issues include (a) privacy concerns of the patients to share their data, (b) unavailability of high-end equipment for digital pathology in developing or underdeveloped countries, and (c) different levels of expertise of the pathologists. These problems and issues may be generalized as lack of labelled pathological data in quantity as well as quality.

There are many issues in the publicly provided histopathological datasets. Most often datasets contain information on the count of lymphocytes present in each patch. In contrast to this, information regarding lymphocyte localization i.e. the bounding box coordinates, and shape of segmentation masks are not provided. In most of the cases datasets may contain mixture of different classes like tissue patches with lymphocytes, without lymphocytes, and artifacts. This requires an additional step to separate and tag each class. Additionally, the number of lymphocytes within the image patch varies from one to hundreds of lymphocytes. Thus, manually defining shape of an object in each sample under a microscope is tiresome and slow, which demands an alternate fast solution. Additionally, the non-conformity of lymphocytes with a specific morphology on stained images makes it difficult to generate mask via a single image processing technique. Consequently, there is a need to develop a fast and generalized approach to generate boundary box and segmentation mask, simultaneously for lymphocytes in each patch image.

In this paper, we have developed a tool, lymphocyte annotator to annotate and address the issues associated with the histopathological datasets. In this regard, CD3+ and CD8+ immunohistochemistry (IHC) stained tissue samples were used. Samples images can be seen in Fig. 1

Original patch images were available in RGB format; whereas literature suggests that such images may be deconvolved to a new image known as Diaminobenzidine (DAB) with enhanced target objects. The conventional method used for deconvolution of such images is presented by Ruifrok et al. [1]. In some cases, object is clearly identifiable in original image and in other cases in DAB image. This posed a problem for annotators to perform annotations while observing both images, as changing images from time to time during annotations was tiresome. To the best of our knowledge, the existing state of the art tools do not provide option to view two images simultaneously and annotate one of them while generating multiple segmentation masks automatically using local information of object rather than using whole image. In this work we exploited the aforementioned idea to generate a DAB image for every RGB image.

To address the aforementioned problems, we present a tool with following key features.

1. Display of both the RGB Image and DAB images simultaneously
2. Automated generation of five different variants of segmentation masks using multiple segmentation models along with saving the user's preferred mask
3. Variable or fixed size drawing of bounding boxes using two clicks or single click correspondingly
4. Option to mark an image "Ambiguous" if found not reliable or is stain artifact
5. Labelling of images into Artifact or Normal for classification purpose
6. Annotation output in PASCAL VOC and JSON format

Table I Libraries and modules used in Lymphocyte Annotator

Libraries/packages	Version
Pillow	7.2.0
Tqdm	>=4.3.1
Pandas	>=0.24.1
Numpy	>=1.16.5
Scikit image	0.17.2
tkinter	8.6.10
jinja	~2.11.2
opencv-python	~4.3.0.36
future	~0.18.2

II. EXISTING WORK

There are many commercially available software packages for viewing, editing, and labelling of large number of natural images but are not specifically tailored to deal with histopathological images. Besides, the available histopathology tools, do not allow user to view and annotate images in different modalities as well as generate variants of segmentation masks automatically.

There are some available tools for image annotations, which allow users to draw bounding boxes or objects shapes in images. Visual Geometry Group developed a web-based tool named as VGG Image Annotator to create annotations for three different modes of data (images, video and audio) [2]. This tool provides the segment annotation option for audios and video along with drawing of objects shapes in images. This software generates the annotations in the JSON and CSV formats. Similarly, Torralba et al. presented a web-based annotation tool, LabelMe [3]. It allows to assign semantics to images by defining objects' spatial location along with label. SlideRunner [4] is developed for annotations in whole slide images. This software uses OpenSlide [5] as image loading backend. It allows user to perform annotations as a single point or periphery/boundary of objects. This tool targets gigapixel images but does not accommodate small image patches. Label studio [6] is a web-based, open source labelling and annotation tool released in late 2019. This tool is simple yet powerful along with the support for Multiple bounding box shapes. Visual Object Tagging tool (VoTT) [7] is also an open source annotation and labelling tool for videos and images, which can be run locally. It can be used easily in any modern browser. VoTT provides the support for windows, Linux and OSX. Computer vision annotation tool (CVAT) [8] was developed by Intel. CVAT is an open source, web-based annotation tool with support for both images and videos. It is a browser-based application and only works with Google Chrome browser.

However, most of the tools for labelling medical imaging datasets are either for commercial use or have high priced

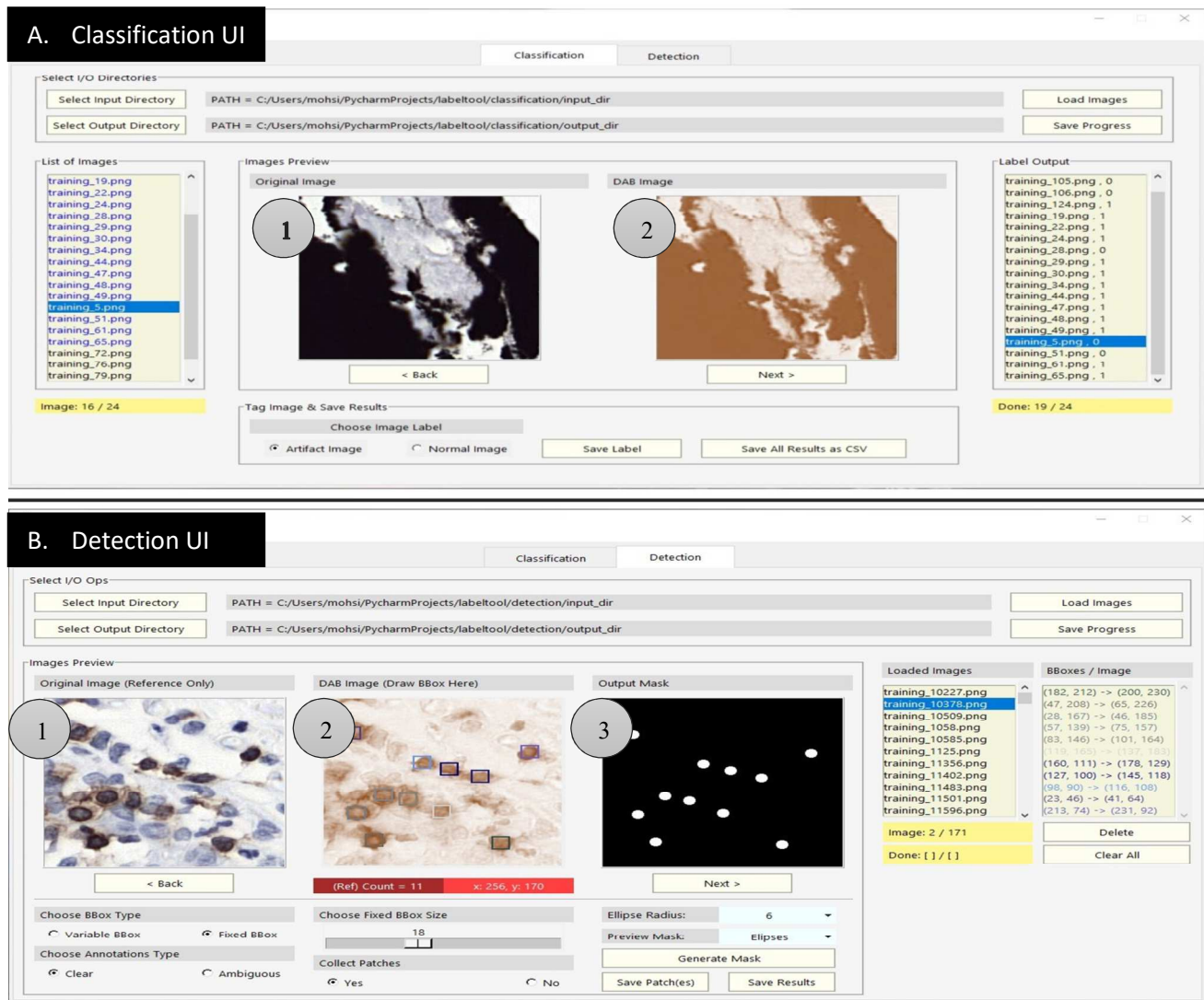


Fig. 2 Lymphocyte Annotator - User Interface (2.A corresponds to Classification Tab whereas 2.B corresponds to the Detection Tab)

subscriptions. In our work, we aimed to design an open source simple and easy to use tool for pathologists to perform annotations with minimum training.

III. LYMPHOCYTE ANNOTATOR

This section discusses the user interface, targeted functionality and different libraries involved in the development of “Lymphocyte Annotator”.

A. Targeted Dataset

The dataset under consideration was released as part of LYSTO’19 Grand Challenge and contained a total of 20,000 images. The task of this challenge was automatic assessment of lymphocytes in tissue sections of several types of cancer [9].

B. Development Environment

Lymphocyte Annotator was developed on a Windows 10 machine with Python 3.7.6 as the programming language in PyCharm Professional 2020.1. For GUI development the tkinter library of python was used. Different image libraries and packages such as scikit-image [10], pillow [11], and opencv-python [12] were used. In order to generate the annotations in the standard PASCAL VOC format, jinja2

template was used. This tool can also be used in Linux environment due to the cross-platform nature of python and tkinter. Details of all libraries involved in the development of tool are provided in the Table 1.

C. User Interface

Fig. 2 depicts the user interface of the Lymphocyte Annotator. It can be clearly seen in the Fig. 2 that there are two different types of tabs in the tool. First tab named “Classification” renders the UI for tasks related to classification between a normal and an artifact image. Whereas the second tab named “Detection” render the UI for tasks related to bounding box and segmentation mask generation. The details of abovementioned two tabs are as follows.

1) Classification Tab

Classification Tab is shown in Fig. 2A. This clearly shows that UI is quite simple and was developed keeping in mind a low learning curve and minimal training. The UI on Classification Tab gives the option to the user to load input directory. The input directory is required to be containing two sub folders: (i) a folder containing DAB images and (ii) the other containing original RGB images. Similarly, there is a

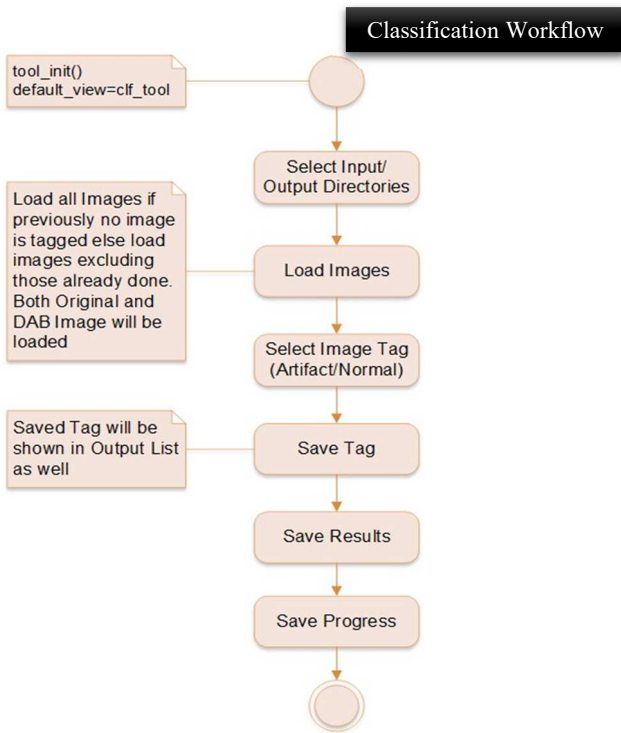


Fig. 3 Classification Tool Workflow - (The flow diagram shows main steps involved in classification workflow using the tool)

button to specify the output directory in which the output file containing assigned labels will be stored. The button “Load

Images” loads the images into the working directory. At the left side of the UI there is a list box containing list of all loaded images and below it is a progress tracker that allows the user to keep track of the number of images and current position. Similarly, on the right most side there is another list box comprising the output in form of “<filename>, <label>”. There is a counter below the list box that keeps track of the images that have been tagged. Two radio buttons allow for the user to tag the image as an Artifact or Normal. The “Save Tag” button tags the image to the “Output List Box”. The “Save All results in csv” button generates the “.csv” file for all tagged images. Before ending a session, user may save progress using the “Save Progress” button. Navigation buttons are available as well. At the center of the UI both DAB and RGB images are displayed

2) Detection Tab

Detection Tab is shown in Fig. 2B. The UI of this tab provides number of options to the user. The UI to load the input and output directories are similar to the Classification Tab. Like Classification Tab there are two list boxes at the right side of the tool. The first list box contains the names of loaded images along with a progress tracker at the bottom. The second list box contains the bounding boxes marked on the image. There are three preview images at the center of UI (shown in Fig. 2B). The panel (1) of Fig. 2B shows the reference image in RGB format, panel (2) illustrates the DAB image where the user is allowed to draw bounding boxes, and the last panel (3) previews the generated segmentation mask. Below the DAB image, boundary box (x, y) coordinates are also visible to the user (Fig. 2B, panel (2)). At the bottom of the tool the user is given multiple options including (i) option to select a variable sized bounding box or a fixed size bounding box, (ii) option to mark an image as clear or

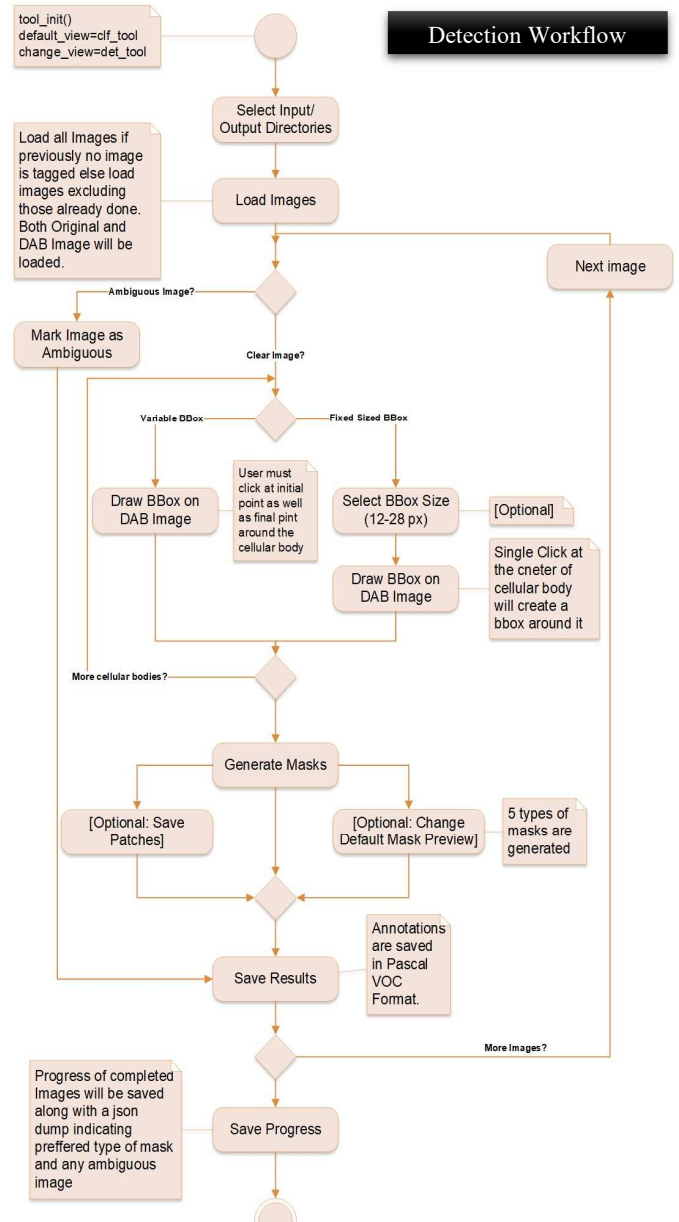


Fig. 4 Detection Tool Workflow - (The flow diagram shows main steps involved in detection workflow using the tool)

ambiguous, (iii) a scale bar to select size of bounding box, (iv) option to save bounding box patches, (v) option to set a radius of circular masks, (vi) option to change the preview of mask type, (vii) option to generate segmentation masks, and (viii) finally saving the results.

D. Targeted Functionality

This section discusses the functionality of the tool from technical perspective. As, mentioned in the previous sections, the tool provides the user with two major functionalities (i) tagging of images between artifacts and normal tissue patches, and (ii) boundary box generation with binary masks. The tagging functionality can be used for the development of classification model, whereas the other functionality: bounding box and mask can be used for training of detection and segmentation models. The tool also provides solution to the problems discussed in the “Introduction” section. It displays original image and corresponding deconvolved image simultaneously, allowing the user to draw bounding

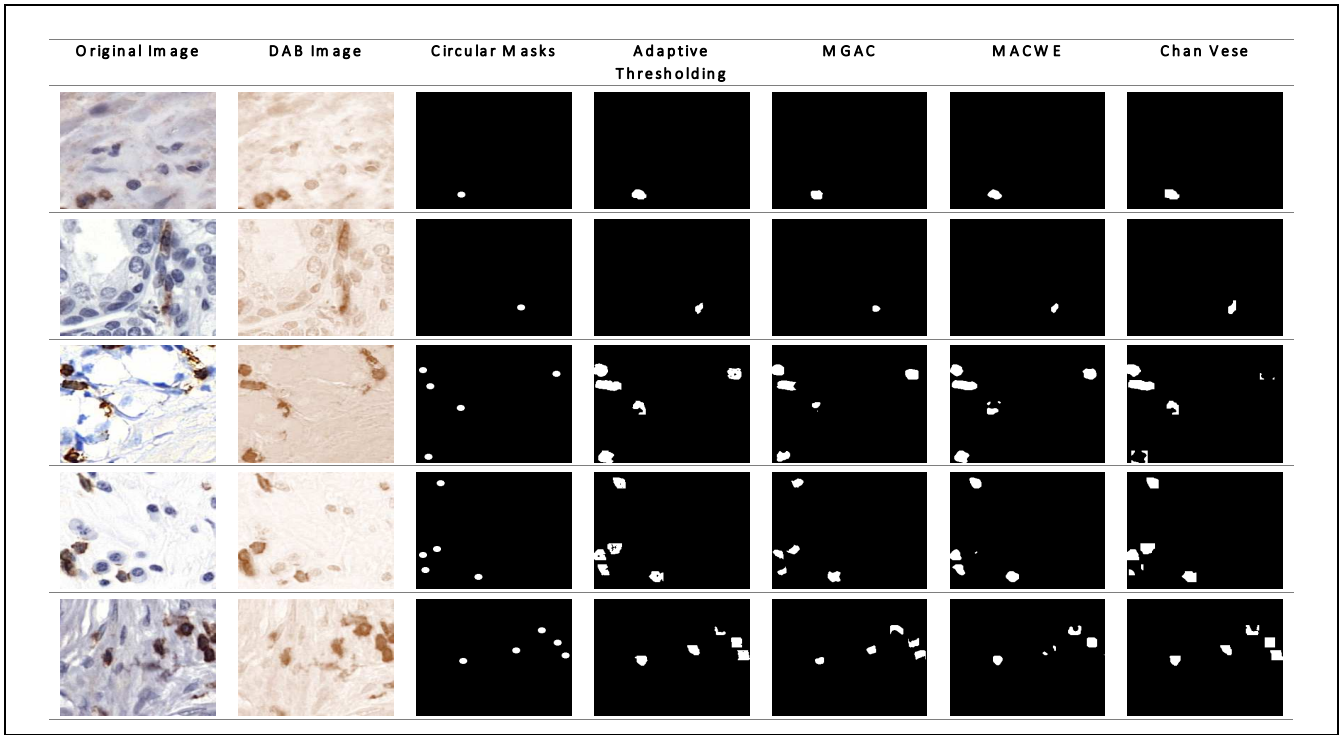


Fig. 5 Binary Mask Outputs – (These masks are generated as a result of the detection workflow)

boxes on one of the images while other remains as a reference image only. Also, to make the annotation task efficient, the tool provides a functionality of drawing fixed size bounding boxes with a single click. The problem of non-specified lymphocyte shape was resolved by generating masks through five different models. These models generate the five different categories of masks that allow the user to select most appropriate segmentation mask or a mixture of two or more for robust training of deep learning models.

1) Classification Workflow

User input in this workflow includes selection of a label using the options presented with Radio Buttons, and then saving the tag using “Save Tag” Button. Both radio buttons have an attached value behind them 0 or 1. After completing the tagging of images the user may save the assigned tags in a csv format using the “Save all Results as CSV” button, and finally save progress using the “Save Progress” button. Saving the progress allows the tool to keep track of images that were tagged in a single session so that there are no repetitions in the next session. Fig. 3 shows workflow of classification tool.

2) Detection Workflow

User input and tool’s output in this workflow is quite extensive as the workflow starts with user’s choice of the type of bounding box either from variable or fixed size bounding box. Variable size bounding box selection requires the user to make two clicks on the opposite corners surrounding the object. The first click yields (x1, y1) and second click yields (x2, y2) coordinates thus drawing a rectangular bounding box around the object. Fixed size bounding box requires a single click at the center of the object. The size can be adjusted from the displayed “scale bar”. Default value of fixed size bounding box is 12 pixels, and a maximum value can be 28 pixels. Each individual bounding box drawn on the DAB image may be deleted using the “Delete” button available on

the right most corner of the UI or all bounding boxes may be cleared out at once using “Clear All” button.

The user is also given the choice to mark an image as “ambiguous” in case of artifact or deformed images, etc. User is also provided with the option to save image patches against the specified boundary boxes. For saving patches user just has to click on the “Save Patches” button and the patches will be saved in output directory under the subdirectory of “patches”. User can also set disk radius for circular masks however the default size is six pixels.

After drawing the bounding boxes, the next step in the workflow is generating segmentation masks. The tool currently utilizes five different methods for generating binary segmentation masks which are named as under.

- Chan Vese Segmentation Model*
- Morphological Active Contours Without Edges Model*
- Morphological Geodesic Active Contours Model*
- Adaptive Thresholding*
- Circular Masks*

After generating masks, the next step in the workflow is to save result which can be done by clicking “Save Result” button.

Finally, the workflow ends by saving the progress using “Save Progress” button. This feature not only allows the tool to keep track of completed annotations and the left over ones but also produces a JSON dump that includes information on the user’s preferred segmentation mask (the masks which is selected for preview) as well as tag of image i.e. ambiguous or clear. Consider Fig. 4 for Detection Workflow.

IV. RESULTS & DISCUSSION

Segmentation masks generated by the tool are depicted in Fig. 5. The results of localized adaptive thresholding and

morphological geodesic active contour (MGAC) [13] model are close to the targeted cellular shapes do not require any post processing step. However, evident from row 3, 4 and 5 of Fig. 5, the output masks obtained using morphological active contours without edges (MACWE) [14] model and Chan Vese [15] model are noisy and therefore, require some post processing steps. Our analysis also suggests that further parameter tuning of these models may solve the problem. As there is no specified representation of lymphocytes in IHC stained images, therefore at times mask generation models fail to generate an appropriate mask. In such cases, lymphocytes can be effectively represented via circular masks. Although circular masks do not represent the exact morphology of lymphocytes accurately; however, it establishes that a body is present at the specified location. We find that using Lymphocyte Annotator lead to a fast annotation process of lymphocyte figures in histopathological images.

V. CONCLUSION AND FUTURE WORK

In this paper, we have developed an annotation tool ‘Lymphocyte Annotator’ for histopathological images. The developed tool can be used to tag the IHC stained histopathological image into specified classes (normal and artifact images). In addition to this, it can effectively develop the bounding boxes and annotation masks in standard PASCAL VOC and JSON format. Currently, the tool is developed for a desktop environment using “Tkinter” library, however we plan to develop a web-based tool with similar core functionality and many other features to make the nature of tool more generic to other datasets and problems as well. The advancements in Deep Convolutional Neural Networks [16] also motivate us for an improvement in the tool by incorporating automated generation of segmentation masks by using localization and contextual information of individual cellular bodies. Some of the features to be included in next version are: (a) output annotations in multiple standard formats, (b) options to tweak parameters of four models applied for generations of masks, (c) deep Learning based single click bounding box generation, and (d) implementing the tool on principles of Software as a Service Architecture and develop a platform for pathologists to take jobs of annotations as well as agreement surveys on already annotated data.

REFERENCES

- [1] A. C. Ruifrok and D. A. Johnston, “Quantification of histochemical staining by color deconvolution,” 2001.
- [2] A. Dutta and A. Zisserman, “The VIA Annotation Software for Images, Audio and Video,” *MM 2019 - Proc. 27th ACM Int. Conf. Multimed.*, pp. 2276–2279, Apr. 2019, doi: 10.1145/3343031.3350535.
- [3] A. Torralba, B. C. Russell, and J. Yuen, “LabelMe: Online image annotation and applications,” in *Proceedings of the IEEE*, 2010, vol. 98, no. 8, pp. 1467–1484, doi: 10.1109/JPROC.2010.2050290.
- [4] M. Aubreville, C. Bertram, R. Klopffleisch, and A. Maier, “SlideRunner: A tool for massive cell annotations in whole slide images,” in *Informatik aktuell*, Feb. 2018, no. 211279, pp. 309–314, doi: 10.1007/978-3-662-56537-7_81.
- [5] M. Satyanarayanan, A. Goode, B. Gilbert, J. Harkes, and D. Jukic, “OpenSlide: A vendor-neutral software foundation for digital pathology,” *J. Pathol. Inform.*, vol. 4, no. 1, p. 27, 2013, doi: 10.4103/2153-3539.119005.
- [6] M. Tkachenko, M. Malyuk, N. Shevchenko, A. Holmanyuk, and N. Liubimov, “Label Studio: Data labeling software,” 2020. <https://github.com/heartexlabs/label-studio> (accessed Oct. 09, 2020).
- [7] “microsoft/VoTT: Visual Object Tagging Tool: An electron app for building end to end Object Detection Models from Images and Videos.” <https://github.com/microsoft/VoTT> (accessed Oct. 09, 2020).
- [8] “opencv/opencv: Powerful and efficient Computer Vision Annotation Tool (CVAT).” <https://github.com/opencv/opencv> (accessed Oct. 09, 2020).
- [9] F. Ciompi, Y. Jiao, and J. van der Laak, “Lymphocyte Assessment Hackathon (LYSTO),” Oct. 2019, doi: 10.5281/ZENODO.3513571.
- [10] S. Van Der Walt *et al.*, “Scikit-image: Image processing in python,” *PeerJ*, vol. 2014, no. 1, 2014, doi: 10.7717/peerj.453.
- [11] “Pillow — Pillow (PIL Fork) 7.2.0 documentation.” <https://pillow.readthedocs.io/en/stable/index.html> (accessed Oct. 09, 2020).
- [12] “opencv-python · PyPI.” <https://pypi.org/project/opencv-python/> (accessed Oct. 09, 2020).
- [13] P. Marquez-Neila, L. Baumela, and L. Alvarez, “A morphological approach to curvature-based evolution of curves and surfaces,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 36, no. 1, pp. 2–17, Jan. 2014, doi: 10.1109/TPAMI.2013.106.
- [14] T. Chan and L. Vese, “An active contour model without edges,” in *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 1999, vol. 1682, pp. 141–151, doi: 10.1007/3-540-48236-9_13.
- [15] P. Getreuer, “Chan-Vese Segmentation,” *Image Process. Line*, vol. 2, pp. 214–224, Aug. 2012, doi: 10.5201/ipol.2012.g-cv.
- [16] A. Khan, A. Sohail, U. Zahoor, and A. S. Qureshi, “A survey of the recent architectures of deep convolutional neural networks,” *Artif. Intell. Rev.*, vol. 53, no. 8, pp. 5455–5516, Apr. 2020, doi: 10.1007/s10462-020-09825-6.